

Spekforge

Reconstruction of API specifications
from Documentation with LLMs

Technical Documentation of project

Author: Iván Santiago Lastra Romero

Repository: Spekforge

Date: July 9, 2026

Project - pipeline of OpenAPI specification
reconstruction with automatic verification

Contents

1	Introduction	2
1.1	Context and motivation	2
1.2	General objective	2
1.3	The loop of verification	2
1.4	Scope and Approach	3
1.4.1	Verification with graceful degradation	3
1.4.2	Target API with ground truth	3
1.5	General architecture of the system	3
1.5.1	Stages of work	3
1.5.2	The intermediate model and repository	4
1.6	Technological Stack	4
1.7	Document structure	4
2	Phase 0 - Staging of Project	5
2.1	Phase objective	5
2.2	File structure	5
2.3	Background configuration	5
2.3.1	pyproject.toml	5
2.3.2	.gitignore	6
2.3.3	Dependencies installation	7
2.4	Intermediate model	7
2.4.1	ParamSpec	8
2.4.2	SchemaSpec	8
2.4.3	EndpointSpec	9
2.5	Deliverable Validation	9
2.5.1	Tests	9
2.5.2	Execution	10

Chapter 1

Introduction

1.1 Context and motivation

Much of the API documentation is written in natural language and is available in formats designed to be read by people, not machines. This makes automation difficult, since the information is often scattered across HTML portals, PDF documents, manually exported Postman collections, or incomplete OpenAPI specifications that are no longer up to date with respect to the actual behavior of the service. In many cases, there isn't even a formal specification.

Reconstructing this information manually is a slow, repetitive process prone to human error. Therefore, automating this task is essential, as is having a tool that facilitates the generation and maintenance of up-to-date specifications.

This project automates this reconstruction process by using a local language model capable of extracting the structure of an OpenAPI specification from unstructured text documentation. However, the central aspect of the design is to strike a balance between the system's ability to extract information and its reliability, avoiding blind reliance on the responses generated by the language model. To this end, validation mechanisms are incorporated that allow for verifying the consistency and quality of the generated specifications before accepting them as the final result.

1.2 General objective

Build a system that functions as a tool capable of processing API documentation, regardless of its source format whether HTML, PDF, Markdown, Postman collections, or partial OpenAPI specifications and reconstruct a formal, verifiable model of the API in the form of a valid OpenAPI 3.x specification. Based on this specification, the system automatically generates three reusable artifacts, including a client SDK, a mock server, and a contract testing suite, thereby facilitating the integration, validation, and use of the API.

1.3 The loop of verification

Due to the phenomenon of hallucinations in language models, the process of transforming documentation into an OpenAPI 3.x specification can produce incorrect results, such as the creation of nonexistent endpoints, the erroneous assignment of data types, or the duplication of schemas. In this context, the project's goal is to implement mechanisms that detect and mitigate these inconsistencies using structural validation techniques, rule-based linting, and dynamic probing when access to the actual API is available. The discrepancies identified during these stages are used as feedback to iteratively refine the generated specification, increasing its accuracy and consistency with the service's actual behavior.

1.4 Scope and Approach

The project is conceived as a personal endeavor and is based on two preliminary design decisions that guide its implementation and execution.

1.4.1 Verification with graceful degradation

During the project’s execution, an additional dynamic probing module is developed, whose function is to compare the generated OpenAPI specification with the API’s behavior during execution, provided that a sandbox environment is available for testing. This process allows for a more representative fidelity metric by evaluating the correspondence between the reconstructed specification and the service’s actual behavior.

In cases where a sandbox environment is not available, the verification process relies exclusively on structural validation and linting mechanisms, ensuring that the specification complies with the syntactic requirements and best practices defined for OpenAPI, although without validating its conformity with the actual implementation of the API.

1.4.2 Target API with ground truth

An API with an official, publicly available OpenAPI specification such as those for Swagger Petstore, Stripe, or GitHub is selected as the evaluation target. During the reconstruction process, the system is provided only with the API documentation, while the official OpenAPI specification is deliberately hidden to prevent it from influencing the generated results.

Once the reconstruction is complete, the resulting specification is compared with the official specification, which serves as the ground truth. This comparison allows for the calculation of a fidelity metric that quantifies the degree of correspondence between the reconstructed specification and the original, providing an objective measure of the system’s accuracy.

1.5 General architecture of the system

1.5.1 Stages of work

The system is structured as a five-stage pipeline. An intermediate model is exchanged between each stage, this model is implemented using a set of Pydantic models that act as a data contract between the system’s various components. In this way, each stage consumes the model generated by the previous stage and produces the model for the next stage, without needing to know the internal implementation details of the other components.

1. **Ingestion and normalization** - This stage composes from the raw source and conversion to tagged fragments for each candidate endpoint
2. **LLM extraction** - Each fragment is extracted, with a structured output, that in this project is an `EndpointSpec`.
3. **Assembly** - The loose `EndpointSpec` merge in a document OpenAPI 3.x, deduplicating schemas and resolving references
4. **Verification** - Structural validation, linting, and dynamic probing.
5. **Generation** - From the verified spec it produces the SDK client, the mock server and the tests suite

1.5.2 The intermediate model and repository

The repository structure reflects the stages of the pipeline (`ingest/`, `extract/`, `assemble/`, `verify/`, `generate/`) plus a `models/` module that contains the shared contract among the different stages and processes of the pipeline. The goal of this is to keep each module testable in isolation.

1.6 Technological Stack

Stage	Tools
Ingest	<code>markitdown</code> , direct JSON parsers
Extraction	LLM with forced structured output, Chroma/Qdrant (RAG)
Assembly	Serialization Pydantic → OpenAPI 3.x
Verification	<code>openapi-spec-validator</code> , Spectral, Schemathesis
Generation	<code>openapi-generator-cli</code> , Prism, Schemathesis/Dredd
Orchestration	Custom state machine

Table 1.1: Technological stack per pipeline stage

1.7 Document structure

This document is organized according to the project’s development phases, with each chapter corresponding to a phase; it documents the phase’s objective, the design decisions made, the relevant code, and evidence that the phase deliverable has been completed. This chapter covers exclusively **Phase 0 - Staging**; the subsequent phases (Ingest, Extraction, Assembly, Verification, and Generation) are documented in subsequent chapters as they are completed.

Chapter 2

Phase 0 - Staging of Project

2.1 Phase objective

As the first stage of development, the repository's base structure is established and the data contract is defined using the intermediate model, before implementing the system's business logic. This design decision allows all stages of the pipeline to be developed using a previously specified and consistent communication interface.

Defining the contract early on ensures that components can evolve independently, provided they adhere to the agreed-upon data model.

2.2 File structure

The project structure is based on packages, each of which reflects one of the project phases and can be tested individually. The following shows the organization of the repository in its main folder:

```
1 src/spekforge/  
2   models/      # shared intermediate model  
3   ingest/      # Phase 1 - Ingest and normalization  
4   extract/     # Phase 2 - LLM extraction  
5     rag/       # Module of RAG for extended documentation  
6   assemble/    # Phase 3 - Final OpenAPI assembly  
7   verify/      # Phase 4a - static and dynamic validation  
8   generate/    # Phase 4b - SDK, mock server, contract tests  
9   orchestrator/ # state proper machine  
10  evaluation/  # ground-truth metrics
```

Listing 2.1: Structure of `src/spekforge`

2.3 Background configuration

The project uses Python 3.12 with `uv`

2.3.1 `pyproject.toml`

```
1 [project]  
2 name = "spekforge"  
3 version = "0.1.0"  
4 description = "Reconstruction of OpenAPI specifications from informal  
5   documentation, with loop verification"  
6 readme = "README.md"  
7 requires-python = ">=3.12"
```

```

7 dependencies = [
8     "pydantic>=2.7",
9     "pyyaml>=6.0",
10    "markdown>=0.0.1a3",
11    "openapi-spec-validator>=0.7",
12    "schemathesis>=3.30",
13    "httpx>=0.27",
14    "ollama>=0.3",
15    "instructor>=1.4",
16    "python-dotenv>=1.0",
17    "typer>=0.12",
18 ]
19
20 [project.optional-dependencies]
21 rag = [
22     "chromadb>=0.5",
23 ]
24
25 [dependency-groups]
26 dev = [
27     "pytest>=8.0",
28     "pytest-cov>=5.0",
29     "ruff>=0.5",
30 ]
31
32 [build-system]
33 requires = ["hatchling"]
34 build-backend = "hatchling.build"
35
36 [tool.pytest.ini_options]
37 testpaths = ["tests"]

```

Listing 2.2: Project dependencies and config

A local execution solution was chosen as the language model provider in order to avoid the costs associated with using cloud services and to ensure the tool's availability throughout the development and evaluation process. Specifically, **Ollama** was selected as the model execution environment, complemented by Instructor, a library that allows for validating generated responses against Pydantic models and performing automatic retries when the output does not conform to the expected structure.

This design decision provides a fully local, reproducible experimentation environment that is independent of external services.

2.3.2 .gitignore

The following are the exclusions defined for the project's version control. These include files and directories generated during development, dependencies, temporary artifacts, and other elements that are not part of the source code and are therefore excluded from the repository according to the rules set forth in the .gitignore file.

```

1 # Virtual Environment
2 .venv/
3 venv/
4
5 #uv
6 uv.lock.tmp
7
8 #Python
9 __pycache__/
10 *.py[cod]
11 *.egg-info/

```

```

12 .eggs/
13 build/
14 dist/
15
16 # testing and coverage
17 .pytest_cache/
18 .coverage
19 htmlcov/
20 .ruff_cache/
21 .mypy_cache/
22
23 # env files
24 .env
25
26 # outputs generated by pipeline
27 data/outputs/*
28 !data/outputs/.gitkeep
29
30 # Ground - truth data
31 tests/fixtures/*/ground_truth/*
32 !tests/fixtures/*/ground_truth/.gitkeep
33
34 # IDE
35 .vscode/
36 .idea/
37
38 # OS
39 .DS_Store

```

Listing 2.3: version control exclusions

For the purpose of testing, the official OpenAPI specification for the target API is deliberately excluded from the inputs provided to the system. By keeping it hidden during pipeline execution, we ensure that the reconstruction is performed solely based on the available documentation and not through direct inspection of the official specification.

This decision ensures that the comparison performed at the end of the process constitutes an objective evaluation of the system’s ability to reconstruct the specification, avoiding biases resulting from prior access to the reference document.

2.3.3 Dependencies installation

To install the dependencies, the command `uv sync` was executed, creating `uv.lock` and the virtual environment `.venv/`, so that the components could be tested properly. Figure 2.1 below shows the resolution and installation of dependencies without errors



```

(spekforge) ~ /Spekforge git-[main]- uv sync
Resolved 125 packages in 4ms
Checked 89 packages in 10ms

```

Figure 2.1: Command output of `uv sync` completing dependency resolution

2.4 Intermediate model

To build the intermediate model, three Pydantic classes were defined, which are distributed as `ParamSpec`, `SchemaSpec`, and `EndpointSpec`, in that order of dependency.

2.4.1 ParamSpec

```

1 from enum import Enum
2 from pydantic import BaseModel
3
4 class ParamLocation(str, Enum):
5     PATH = "path"
6     QUERY = "query"
7     HEADER = "header"
8     COOKIE = "cookie"
9
10 class ParamSpec(BaseModel):
11     name: str
12     location: ParamLocation
13     required: bool = False
14     type: str = "string"
15     description: str | None = None
16     example: str | None = None

```

Listing 2.4: src/spekforge/models/param.py

In this case, `ParamLocation` was defined as an `Enum`, rather than a free-form text string (`str`), in order to restrict the language model, during Phase 2, to selecting only values belonging to a previously defined and valid set. When generating the JSON schema using (`model_json_schema()`), this enumeration translates into an explicit restriction within the *JSON Schema* used during the *function calling*.

2.4.2 SchemaSpec

```

1 from pydantic import BaseModel, Field
2
3 class SchemaSpec(BaseModel):
4     name: str | None = None
5     type: str = "object"
6     format: str | None = None
7     description: str | None = None
8     properties: dict[str, "SchemaSpec"] = Field(default_factory=dict)
9     required: list[str] = Field(default_factory=list)
10    items: "SchemaSpec | None" = None
11    enum: list[str] | None = None
12    ref: str | None = None
13
14 SchemaSpec.model_rebuild()

```

Listing 2.5: src/spekforge/models/schema.py

The `SchemaSpec` model has a recursive structure, the `properties` attribute corresponds to a set of objects of the same `SchemaSpec` type, while the `items` field represents the schema of the elements contained in an array. This definition allows for the modeling of nested data structures, such as a `Pet` object that contains a list of `Tag` objects, while preserving the hierarchy of the OpenAPI specification.

Due to this self-reference, it is necessary to call the `model_rebuild()` method once the class definition is complete. This procedure resolves references to the class itself, since, during the evaluation of the `SchemaSpec` body, the corresponding symbol has not yet been fully defined.

The `ref` field is intended for Phase 3; `assemble/dedup.py` will use it to indicate that a schema has already been registered in the components under the specified name. This is done to prevent the LLM from describing the same object in slightly different ways in different sections of the documentation.

2.4.3 EndpointSpec

```

1 from enum import Enum
2 from pydantic import BaseModel, Field
3 from spekforge.models.schema import SchemaSpec
4 from spekforge.models.param import ParamSpec
5
6 class HTTPMethod(str, Enum):
7     GET= "get"
8     POST= "post"
9     PUT= "put"
10    DELETE= "delete"
11    HEAD= "head"
12    OPTIONS= "options"
13    PATCH= "patch"
14
15 class AuthType(str, Enum):
16     NONE= "none"
17     API_KEY="api_key"
18     BEARER="bearer"
19     BASIC="basic"
20     OAUTH2="oauth2"
21
22 class EndpointSpec(BaseModel):
23     method: HTTPMethod
24     path: str
25     summary: str | None = None
26     description: str | None = None
27     parameters: list[ParamSpec] = Field(default_factory=list)
28     request_body: SchemaSpec | None = None
29     responses: dict[str, SchemaSpec] = Field(default_factory=dict)
30     auth: AuthType = AuthType.NONE
31     errors: list[str] = Field(default_factory=list)

```

Listing 2.6: src/spekforge/models/endpoint.py

EndpointSpec is the central component of the intermediate model, so Phase 2 is responsible for populating the model for each endpoint detected in the documentation, and Phase 3 uses this to build the final model with the specified configuration. The **responses** field is modeled directly as an extension of `dict[str, SchemaSpec]`, with the HTTP status code as the key, mapping directly to the response block in the specification. Both **request_body** and **auth** are optional modules that are populated since not all endpoints have explicit authentication or a request body.

2.5 Deliverable Validation

2.5.1 Tests

```

1 from spekforge.models.endpoint import EndpointSpec, HTTPMethod, AuthType
2 from spekforge.models.param import ParamSpec, ParamLocation
3 from spekforge.models.schema import SchemaSpec
4
5 def test_endpoint_spec_minimal():
6     endpoint = EndpointSpec(method=HTTPMethod.GET, path="/pets/{id}")
7
8     assert endpoint.method == HTTPMethod.GET
9     assert endpoint.path == "/pets/{id}"
10    assert endpoint.auth == AuthType.NONE
11    assert endpoint.parameters == []
12

```

```

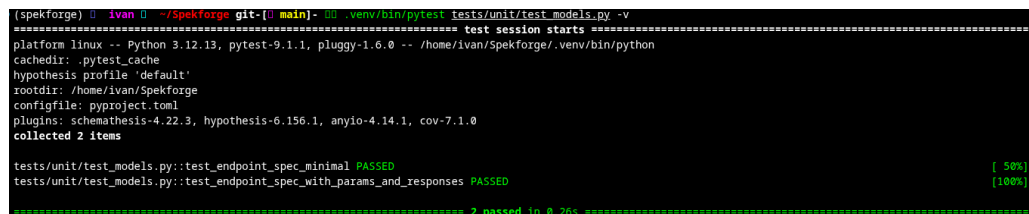
13 def test_endpoint_spec_with_params_and_responses():
14     pet_schema = SchemaSpec(
15         name="Pet",
16         properties={
17             "id": SchemaSpec(type="integer"),
18             "name": SchemaSpec(type="string"),
19         },
20         required=["id", "name"],
21     )
22     id_param = ParamSpec(name="id", location=ParamLocation.PATH, required=True,
23                          type="integer")
24
25     endpoint = EndpointSpec(
26         method=HTTPMethod.GET,
27         path="/pets/{id}",
28         parameters=[id_param],
29         responses={"200": pet_schema},
30     )
31
32     assert endpoint.parameters[0].name == "id"
33     assert endpoint.responses["200"].properties["name"].type == "string"

```

Listing 2.7: tests/unit/test_models.py

The first test covers a minimum case required by the phase deliverable. This test verifies the functionality of the intermediate model; in addition, a second test is performed to verify the nesting of ParamSpec and SchemaSpec within EndpointSpec, including the recursion of SchemaSpec.properties.

2.5.2 Execution



```

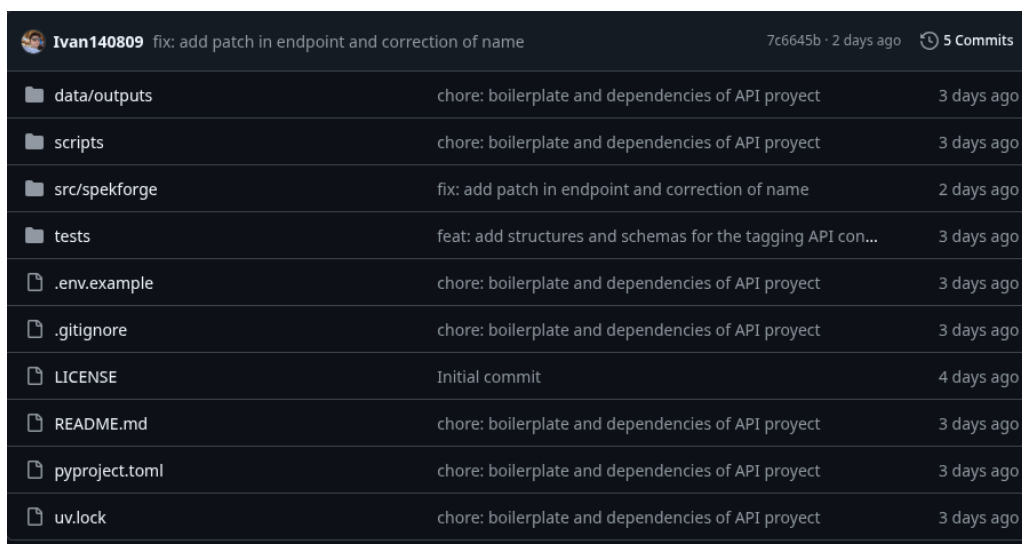
(spekforge) U Ivan U ~/Spekforge git-[U main]- U .venv/bin/pytest tests/unit/test_models.py -v
===== test session starts =====
platform linux -- Python 3.12.13, pytest-9.1.1, pluggy-1.6.0 -- /home/ivan/Spekforge/.venv/bin/python
cachedir: .pytest_cache
hypothesis profile 'default'
rootdir: /home/ivan/Spekforge
configfile: pyproject.toml
plugins: schemathesis-4.22.3, hypothesis-6.156.1, anyio-4.14.1, cov-7.1.0
collected 2 items

tests/unit/test_models.py::test_endpoint_spec_minimal PASSED [ 50%]
tests/unit/test_models.py::test_endpoint_spec_with_params_and_responses PASSED [100%]

===== 2 passed in 0.26s =====

```

Figure 2.2: pytest output showing the two passing tests of the intermediate model



 Ivan140809	fix: add patch in endpoint and correction of name	7c6645b · 2 days ago	 5 Commits
data/outputs	chore: boilerplate and dependencies of API proyect	3 days ago	
scripts	chore: boilerplate and dependencies of API proyect	3 days ago	
src/spekforge	fix: add patch in endpoint and correction of name	2 days ago	
tests	feat: add structures and schemas for the tagging API con...	3 days ago	
.env.example	chore: boilerplate and dependencies of API proyect	3 days ago	
.gitignore	chore: boilerplate and dependencies of API proyect	3 days ago	
LICENSE	Initial commit	4 days ago	
README.md	chore: boilerplate and dependencies of API proyect	3 days ago	
pyproject.toml	chore: boilerplate and dependencies of API proyect	3 days ago	
uv.lock	chore: boilerplate and dependencies of API proyect	3 days ago	

Figure 2.3: Repository file tree on GitHub, showing the complete structure of `src/spekforge/` at the close of Phase 0